

MODULE 5 · STORAGE

StorageClasses & Dynamic Provisioning

Stop hand-provisioning volumes — let a StorageClass do it for you

Lab 5.2 made you create the PV by hand — that doesn't scale

In 5.2 you wrote a **PersistentVolume** yourself, sized to match one PVC. That's fine for one volume — it falls apart the moment ten teams each want their own claim.

A **StorageClass** tells Kubernetes how to make storage on demand. A matching PV appears automatically the moment a PVC asks for one — nobody writes it.

BY THE END YOU CAN

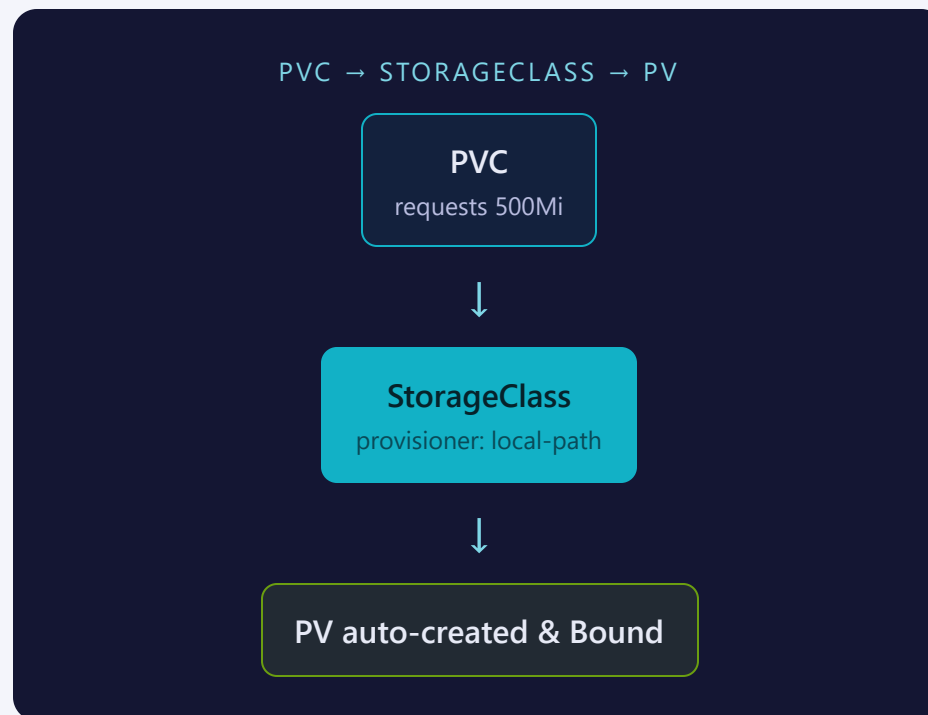
- Read a StorageClass's key fields
- Explain what the default class does
- Create a PVC that provisions its own PV
- Explain why WaitForFirstConsumer delays binding

CONCEPT

A StorageClass is a template, not a volume

A StorageClass holds no data. It just names a **provisioner** — the plugin that knows how to create storage — plus a policy for how that storage behaves.

When a PVC asks for a class, the provisioner creates a matching PV and binds it, automatically. Nobody writes a PV manifest.



Three fields decide the behaviour

FIELD	WHAT IT CONTROLS
<code>provisioner</code>	Which plugin creates the volume, e.g. <code>rancher.io/local-path</code>
<code>reclaimPolicy</code>	<code>Delete</code> — PV removed with the PVC, or <code>Retain</code> — kept
<code>volumeBindingMode</code>	<code>Immediate</code> — provision now, or <code>WaitForFirstConsumer</code> — provision when a Pod needs it

THE DEFAULT CLASS

One class carries the annotation `storageclass.kubernetes.io/is-default-class: "true"`. A PVC that omits `storageClassName` uses it automatically.

No spec: on the class — and the PVC that names it

storageclass.yaml

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: local-path
provisioner: rancher.io/local-path # creates the volume
reclaimPolicy: Delete # PV tied to the PVC
volumeBindingMode: WaitForFirstConsumer # wait for a Pod
```

pvc.yaml

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: dynamic-pvc-explicit
  namespace: training
spec:
  storageClassName: local-path # names the class above
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 250Mi
```

No spec: block on the StorageClass — its fields sit at the top level. Omit storageClassName on the PVC and the default class is used instead.

Take the default, or name the class

OMIT STORAGECLASSNAME	SET STORAGECLASSNAME EXPLICITLY
Admission fills in the default class for you	You control exactly which class binds
"Just give me the default disk"	Needed once more than one class exists

THE BRIDGE TRICK

A server-side dry run runs the same admission step as a real apply — including the default-class mutation — so you see which class you'll actually get before committing:

```
$ kubectl apply -f pvc.yaml --dry-run=server -o yaml \
  | grep storageClassName
```

SEE IT

Pending is not broken — it's waiting for a consumer

With `volumeBindingMode: WaitForFirstConsumer`, `kubectl get pvc` shows the claim stuck at Pending right after creation. That's expected — no PV exists yet.

The moment a Pod mounts the PVC and gets scheduled, the provisioner creates the PV on that node and the claim flips to Bound.

```
$ kubectl get pvc dynamic-pvc-default -n training
NAME                                STATUS
dynamic-pvc-default                Pending    # no consumer yet

# after a Pod mounts it:
dynamic-pvc-default                Bound     pvc-3f2a...
```

When does the PV actually get created?

MODE	PV CREATED	BEST FOR
Immediate	As soon as the PVC exists	Storage with no topology constraint
WaitForFirstConsumer	When a Pod using the PVC is scheduled	Storage tied to a specific node — the common case

WATCH

`storageClassName: ""` turns dynamic provisioning off entirely. That PVC waits forever for a pre-created PV — Lab 5.2's world, not this one.

Where people trip

- **Panicking at Pending.** Check `volumeBindingMode` first — `WaitForFirstConsumer` makes it normal.
- **Assuming "only one class" means default.** Check the `is-default-class` annotation — none may be marked.
- `storageClassName: ""` **isn't "unset."** It explicitly disables dynamic provisioning.
- **Deleting a PVC on a Delete class** also deletes the PV and its data — use `Retain` if you need to keep it.

Commands to keep at hand

```
$ kubectl get sc                # list classes, (default) flag
$ kubectl describe sc NAME      # provisioner, reclaimPolicy, mode
$ kubectl get pvc -n NS         # Pending vs Bound
$ kubectl get pv                # the auto-created PVs
$ kubectl apply -f pvc.yaml --dry-run=server -o yaml # preview the resolved class
```

Tip: a claim stuck Pending is normal under WaitForFirstConsumer — don't "fix" it before a Pod tries to mount it.

RECAP

StorageClasses in one breath

- A StorageClass names a provisioner + policy, not a volume
- A PVC without storageClassName gets the default class
- Dynamic provisioning creates the PV for you — no manual manifest
- WaitForFirstConsumer defers provisioning until a Pod is scheduled

HANDS-ON

Now run Lab 5.3
[labs/5.3/](#)

Next: [5.4 — PVCs in Deployments & StatefulSets](#)